Big Data Bowl 2027: Slant Route Plan

Nov 11, 2026 · @ava

The project in one paragraph

Our question: does how a wide receiver brakes and cuts on the slant route at the Combine predict how open he gets on slant routes in real NFL games?

Think of a slant like a car taking a sharp turn. Some drivers slam the brakes and wobble; others slow down smoothly and zoom out of the turn. The Combine sensors recorded every receiver's "turn" 10 times a second. The NFL data tells us how many yards of space each receiver had from the nearest defender when the ball was thrown. We connect the two.

What we have to work with:

- **Combine side:** 107 receivers ran the slant drill (`SLANT_ROUTE_LEFT`). Most ran it once, which gives about 47 sensor readings (about 4.7 seconds) per player.
- **NFL side:** 100 of those receivers ran 2,674 slants in NFL games with a separation number. The middle player ran 15 slants. A quarter of players ran 5 or fewer, so small samples are our biggest danger.
- **Typical NFL slant separation:** 2.2 yards (middle value), ranging from 0.2 to 12 yards.

The big picture: 5 steps

The whole project is five steps. Steps 1 and 2 happen at the same time, one person each. Steps 3 to 5 we do together.

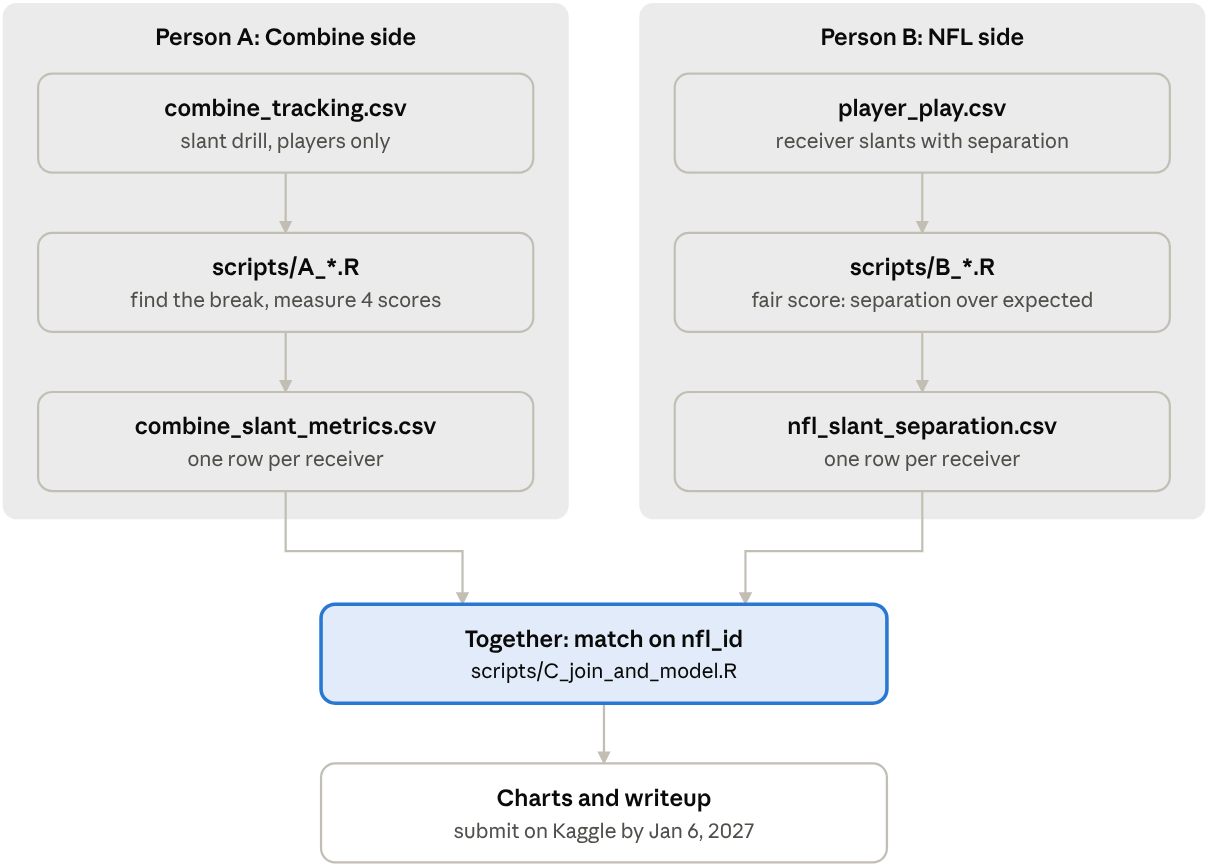
1. **Measure the Combine slant (Person A).** Turn each receiver's sensor dots into a few simple scores, like "how hard did he brake?" and "how fast did he speed back up?"
2. **Measure NFL slant separation (Person B).** Turn each receiver's NFL slants into one fair "how open does he get?" score. Fair means we adjust for things that aren't his fault, like how far back the defender started.
3. **Connect them (together).** Match the two tables by player ID and check whether good brakers get more open.
4. **Make the charts (together).** Fewer than 10 charts or tables total, so each one has to earn its spot.

5. **Write it up (together).** Up to 2,000 words, plus a public Kaggle notebook with our code.

How we split the work

Person A owns the Combine side, Person B owns the NFL side, and the two only meet at one file. Each of you can work at your own pace without waiting for the other or touching each other's code.

Each person owns one lane; the lanes only meet at nfl_id



who owns what · 2 lanes, 1 meeting point

Each lane ends in one CSV file. The joining step only starts once both files exist, and both of you are there for it.

Person A: the Combine side

Your job: turn each receiver's Combine slant into a few brake-and-cut scores, one row per player.

Files only you touch: anything in `scripts/` starting with `A_` (for example `scripts/A_01_load_slant.R`), plus the output file `output/combine_slant_metrics.csv`.

- [ ] Load `data/combine_tracking.csv` and keep only rows where `drill_name` is `SLANT_ROUTE_LEFT` **and** `entity_type` is `PLAYER`. (The file also tracks the football itself; those rows say `BALL`.)
- [ ] Draw the path (`x` and `y`) for 5 players so you can see the slant shape with your own eyes.
- [ ] Find **the break**, the moment the player turns. Hint: it's where the direction column `dir` changes the most.
- [ ] Measure 4 scores per player:
  - **Entry speed:** how fast he was going right before the break (yards per second).
  - **Brake:** how hard he slowed down going into the break.
  - **Cut sharpness:** how many degrees he turned, and how quickly.
  - **Burst:** how fast he sped back up in the 1 second after the break.
- [ ] If a player ran the drill more than once, keep his best try (or average them). Write down which one you chose.
- [ ] Check whether your scores just copy the 40-yard dash. Compare them to `forty` and `three_cone` in `combine_results.csv`. If brake looks exactly like the 40 time, it's not telling us anything new.
- [ ] Save `output/combine_slant_metrics.csv` using the exact column names in **The handoff** section below.

Person B: the NFL side

Your job: give each receiver one fair "how open does he get on slants?" score, one row per player.

Files only you touch: anything in `scripts/` starting with `B_` (for example `scripts/B_01_load_slants.R`), plus the output file `output/nfl_slant_separation.csv`.

- [ ] Load `data/player_play.csv` and keep receivers only. Join it to `combine_results.csv` by `nfl_id` and keep rows where `combine_position` is `WR`.
- [ ] Keep only slants: rows where `route_ran` is `SLANT` and `separation_at_pass_forward` is not blank.
- [ ] Make a histogram of separation so you know what "normal" looks like (the middle is about 2.2 yards).
- [ ] **Make it fair.** Some open-ness isn't the receiver's doing. Build a simple regression that predicts separation from things he doesn't control:
  - `cushion`: how far back the defender started.
  - `team_coverage_man_zone`: man coverage (someone follows him) vs zone (defenders guard areas).
  - `receiver_alignment`: where he lined up.
  - `down` and `yards_to_go`.
- [ ] For every slant, calculate **separation over expected** = actual separation minus what the model predicted. Positive means he got more open than the situation explains.
- [ ] Average it per player. **Watch out for small samples:** a player with 3 slants can look great by luck. Either require a minimum (for example 10 slants) or pull small samples toward the average. Write down which one you chose.
- [ ] Save `output/nfl_slant_separation.csv` using the exact column names in **The handoff** section below.

The handoff: two CSV files that snap together

We agree on these column names now, so neither person has to wait for the other. Like two puzzle pieces, the shapes have to match even if we cut them in different rooms. `nfl_id` is the shared edge. Until your real numbers are ready, save a fake version with these exact columns so the joining code can be written early.

Person A saves `output/combine_slant_metrics.csv`, one row per receiver:

Column	What it means	Units
`nfl_id`	Player ID (the shared edge)	number
`attempts`	How many slant tries he had	count
`entry_speed`	Speed right before the break	yards per second

Column	What it means	Units
`brake_decel`	Hardest slowdown going into the break	yards per second²
`cut_angle`	How much he turned at the break	degrees
`burst_1s`	Speed gained in the 1 second after the break	yards per second

Person B saves `output/nfl_slant_separation.csv`, one row per receiver:

Column	What it means	Units
`nfl_id`	Player ID (the shared edge)	number
`n_slants`	How many NFL slants he ran with a separation number	count
`avg_sep`	Plain average separation	yards
`sep_oe`	Average separation over expected (the fair score)	yards

Need a new column? Add it to the end, tell your partner, and update this table. **Never rename or delete a column the other person uses.**

Git rules so we never overwrite each other

The big rule: you only edit files that start with your letter. If two people never touch the same file, Git can't get confused. It's like two roommates who each have their own shelf in the fridge.

File	Owner
`scripts/A_*.R`, `output/combine_slant_metrics.csv`	Person A
`scripts/B_*.R`, `output/nfl_slant_separation.csv`	Person B
`scripts/C_join_and_model.R`	Person A types, Person B reviews
`writeup/A_section.md`	Person A
`writeup/B_section.md`	Person B

File	Owner
`writeup/final.md`, the Kaggle notebook	Person B puts it together at the end
`scripts/01_explore.R`, `.gitignore`	Nobody. Ask first.

Every time you sit down to work:

```
git pull                        # download your partner's newest work first
```

Every time you stop working:

```
git status                      # list what you changed; check it's only YOUR
files
git add scripts/A_*             # stage your files (Person B types scripts/B_*
instead)
git commit -m "Find the break point in slant drill"   # save a snapshot with
a short note
git pull                        # grab anything your partner pushed while you
worked
git push                        # upload your snapshot so your partner can see
it
```

Never do these:

- `git push --force`. It erases your partner's work.
- `git add .` without running `git status` first. It can grab files you didn't mean to share.
- Editing your partner's file "just to fix one thing." Text them instead.

The data stays off GitHub. The `data/` folder is already ignored because the files are 2 GB. Each person downloads their own copy from Kaggle into `data/`.

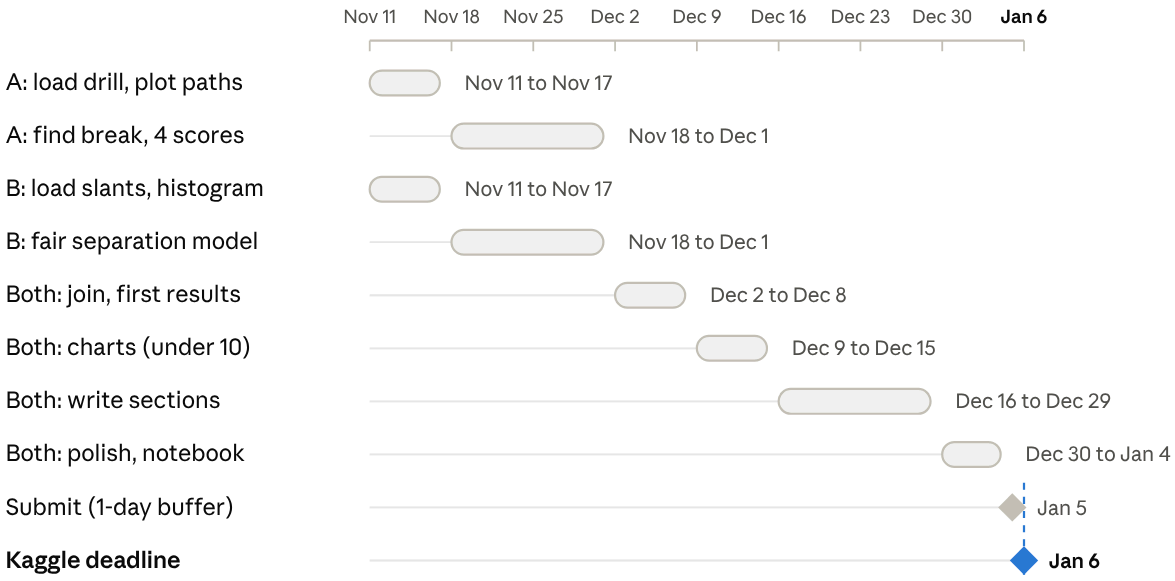
If Git says "conflict": stop, don't guess, and text your partner. It almost always means someone edited the other person's file.

Timeline

Both CSV files are due December 1, so we have 5 weeks together for joining, charts and writing. We aim to submit January 5 to leave a day for mistakes.

Submit Jan 5, one day before the Jan 6 Kaggle deadline

Weekly plan, Nov 2026 to Jan 2027 · deadline is 11:59 PM UTC (6:59 PM Eastern)



week-by-week plan · Nov 11 to Jan 6

If either half runs late, save a fake CSV with the right columns by Dec 1 anyway, so the joining code can still get written.

Words to know

Word	Plain meaning
Slant	A route where the receiver runs a few steps straight, then cuts sharply toward the middle of the field.
Break	The moment in a route when the receiver plants his foot and turns.
Separation	How many yards of space the receiver has from the nearest defender when the ball is thrown.
Separation over expected	Actual space minus the space we'd guess from the situation. Above zero means he beat the odds.
Cushion	How far back the defender starts from the receiver before the play.

Word	Plain meaning
Man / zone coverage	Man: one defender follows him everywhere. Zone: defenders guard areas of the field.
10 Hz	10 readings per second, like a flipbook with 10 pages per second.
`nfl_id`	Each player's ID number. It's how we match the Combine table to the NFL table.
Small sample	Too few plays to trust. 3 good slants could just be luck.
Regression	A math tool that finds how much each thing (like cushion) pushes the result (separation) up or down.
Commit / push / pull	Commit = save a snapshot on your laptop. Push = upload it to GitHub. Pull = download your partner's snapshots.